Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»
Факультет компьютерных систем и сетей
Кафедра информатики

Дисциплина «Операционные системы и системное программирование»

# ОТЧЕТ

к лабораторной работе №6

на тему:

## «Мониторинг изменений значений»

Вариант 6

Выполнил: студент группы _____
Дмитрий Василенко
Проверил: _____

Минск, 2026

# ВВЕДЕНИЕ

Лабораторная работа посвящена служебным механизмам операционной системы: конфигурированию через реестр, журналированию, мониторингу изменений и построению диагностических утилит. Исходное задание сформулировано для Windows, однако выполнение требуется на Ubuntu. Поэтому в работе используется функциональный аналог: иерархическое файловое хранилище, повторяющее структуру учебного раздела реестра, и механизм `inotify`, который, как и `RegNotifyChangeKeyValue()`, позволяет получать уведомления об изменении объектов без активного опроса.

Такой подход сохраняет главное: программа демонстрирует событийное ожидание, повторное чтение состояния, сравнение с предыдущим снимком и протоколирование различий. При этом реальный реестр Windows и системные настройки не затрагиваются.

## ЦЕЛЬ РАБОТЫ

Изучить системные механизмы конфигурирования, мониторинга и журналирования, а также получить практические навыки разработки служебной утилиты, которая:

- работает с иерархическим хранилищем ключей и значений;
- создаёт, изменяет, удаляет и перечисляет значения;
- регистрирует асинхронное наблюдение за изменениями;
- после уведомления повторно читает состояние и определяет разницу;
- ведёт журнал операций с временем, объектом и результатом;
- корректно обрабатывает ошибки и освобождает ресурсы.

# 1. ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Вариант 6. **Мониторинг изменений значений**. Основной механизм — `RegNotifyChangeKeyValue()`.

Требуется разработать утилиту ожидания изменения значений внутри выбранного учебного раздела. Программа должна:

1. открыть раздел с правом уведомления;
2. зарегистрировать асинхронное ожидание;
3. ожидать событие;
4. после уведомления повторно прочитать значения;
5. определить разницу с предыдущим состоянием;
6. продолжить наблюдение.

Эксперименты: изменение одного значения, изменение нескольких значений и удаление значения. Необходимо проверить получение уведомлений и корректное определение итоговых отличий.

Индивидуальные параметры: `STUDENT_ID = 028`, `SEED = 1000 + 28 * 97 = 3716`, `RUN_ID` задаётся при запуске. Учебный ключ:

```
HKEY_CURRENT_USER/Software/OSiSP/Lab06/Student_028/Run_<RUN_ID>
```

## 2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Реестр Windows — иерархическая база конфигурационных данных. Корневые разделы `HKEY_CURRENT_USER`, `HKEY_LOCAL_MACHINE` и другие содержат подключи, а подключи — значения с типами `REG_SZ`, `REG_EXPAND_SZ`, `REG_DWORD`, `REG_QWORD`, `REG_MULTI_SZ`, `REG_BINARY`. Для чтения и изменения используются `RegOpenKeyEx()`, `RegQueryValueEx()`, `RegSetValueEx()`, `RegEnumKeyEx()`, `RegEnumValue()`, `RegDeleteValue()` и другие функции.

Для отслеживания изменений применяется `RegNotifyChangeKeyValue()`. Она регистрирует ожидание изменения ключа или его поддерева и может работать асинхронно: вызывающий поток продолжает работу, а о событии узнаёт через событие синхронизации. После уведомления приложение само повторно читает значения и сравнивает их с ранее сохранённым состоянием.

В Linux аналогом является `inotify`. Программа добавляет watch для каталога ключа, получает файловый дескриптор событий и ожидает его готовности через `poll()`. События `IN_CREATE`, `IN_DELETE`, `IN_MOVED_FROM`, `IN_MOVED_TO`, `IN_CLOSE_WRITE`, `IN_MODIFY`, `IN_DELETE_SELF` позволяют заметить создание, изменение, переименование и удаление значений. Как и в Windows, уведомление лишь сообщает о факте изменения; окончательное состояние нужно прочитать отдельно.

# 3. АРХИТЕКТУРА ПРИЛОЖЕНИЯ

Приложение состоит из четырёх основных частей:

- `main.cpp` — разбор аргументов и выполнение команд;
- `common.h` — типы значений, журнал и вспомогательные функции;
- `registry.cpp` — файловый аналог реестра, проверка прав и снимки;
- `monitor.cpp` — `inotify`, ожидание событий, объединение серий и diff.

Хранилище располагается в каталоге `~/.osip_lab06_registry` (можно изменить параметром `--root`). Каждый ключ — каталог, каждое значение — файл `<имя>.val`. Первая строка файла содержит тип, остальное — экранированные данные. Запись выполняется во временный файл с последующим `rename()`, поэтому читатель видит либо старое, либо новое значение, но не частично записанное.

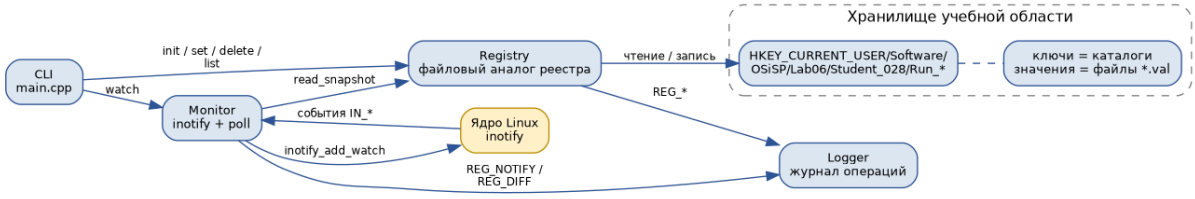



Рисунок 1. Архитектура приложения и связь с ядром Linux

# 4. ОПИСАНИЕ АЛГОРИТМА

Алгоритм монитора:

1. Проверить существование ключа.
2. Создать `inotify`-дескриптор через `inotify_init1()`.
3. Добавить watch для выбранного ключа.
4. Прочитать исходный снимок значений.
5. Создать ready-файл, чтобы внешний сценарий знал о готовности.
6. Ожидать события через `poll()` с учётом тайм-аута.
7. Прочитать пачку событий `inotify` и отфильтровать временные файлы.
8. Выдержать окно `--debounce-ms`, чтобы объединить серию записей.
9. Повторно прочитать снимок и сравнить его с предыдущим.
10. Записать `REG_NOTIFY`, `REG_DIFF` и детали изменений.
11. Повторять до тайм-аута, лимита уведомлений или сигнала.
12. Закрыть дескриптор и записать `MONITOR_STOP`.

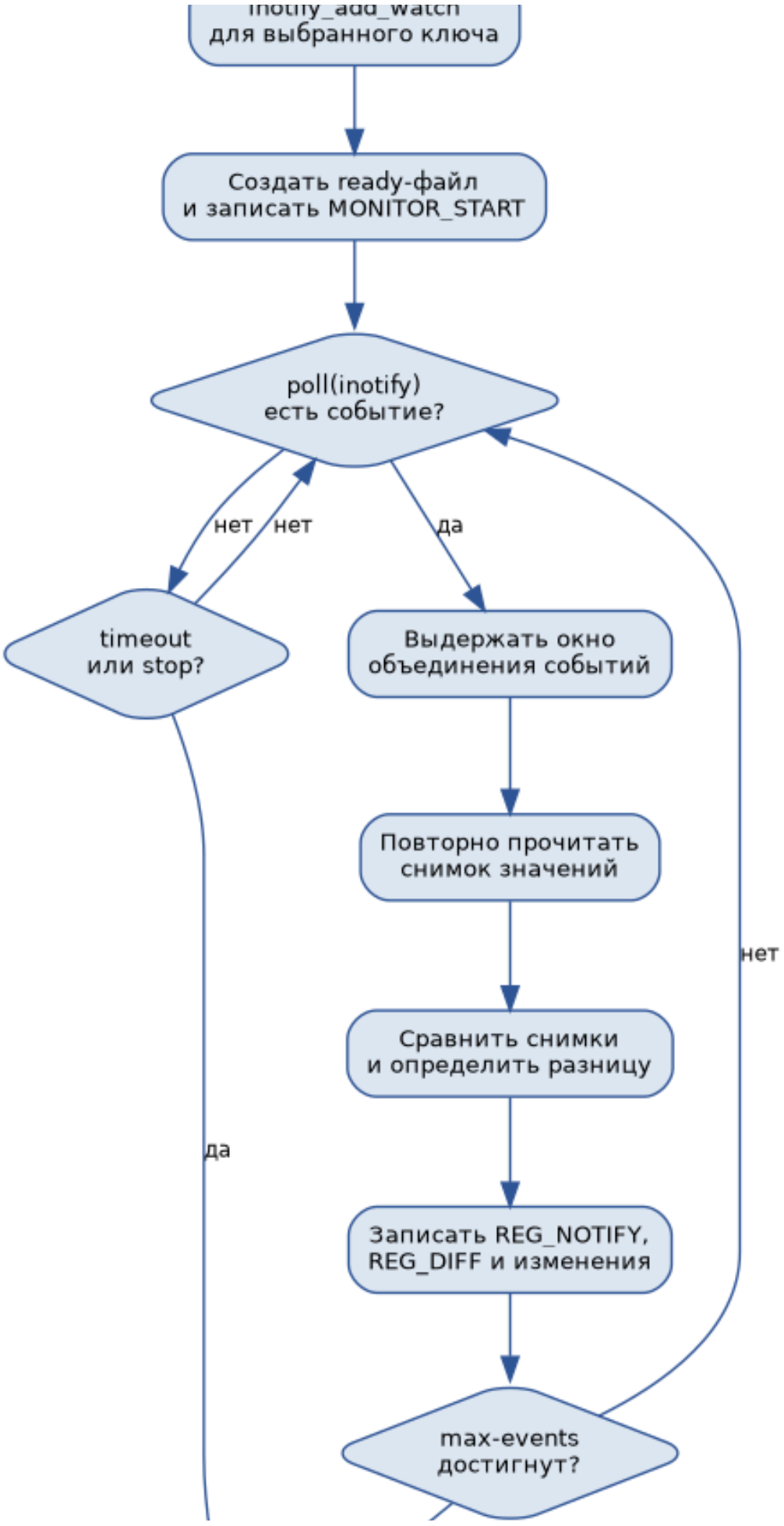



Рисунок 2. Алгоритм работы режима monitor

# 5. ИСПОЛЬЗОВАННЫЕ API

В работе использованы как Windows-понятия, так и конкретные Linux-вызовы.

| **Windows API** | **Linux-аналог в работе** |
|---|---|
| `RegOpenKeyEx()` | проверка каталога ключа |
| `RegCreateKeyEx()` | `create_directories()` |
| `RegQueryValueEx()` | чтение файла `*.val` |
| `RegSetValueEx()` | атомарная запись файла |
| `RegEnumKeyEx()` | обход подкаталогов |
| `RegEnumValue()` | обход файлов значений |
| `RegDeleteValue()` | `unlink()` |
| `RegDeleteKeyEx()` | `remove()` |
| `RegDeleteTree()` | `remove_all()` |
| `RegNotifyChangeKeyValue()` | `inotify_add_watch()` + `poll()` |
| `RegCloseKey()` | закрытие дескрипторов |
| `GetLastError()` | `errno` + `strerror()` |
| `QueryPerformanceCounter()` | `clock_gettime(CLOCK_MONOTONIC)` |

Рисунок 1. Соответствие изучаемых Windows API и Linux-аналогов

Дополнительно используются `open()`, `read()`, `write()`, `fsync()`, `close()`, `rename()`, `unlink()`, `mkstemp()`, `inotify_init1()`, `inotify_add_watch()`, `poll()` и `std::filesystem`.

# 6. СТРУКТУРЫ ДАННЫХ

Основные структуры:

```
struct Value {
    std::string type;
    std::string data;
};

struct Entry {
    std::string key;
    std::string name;
    Value value;
};

struct Snapshot {
    std::map<std::string, Entry> values;
    std::set<std::string> keys;
};

struct Diff {
    std::vector<Entry> added;
    std::vector<Entry> removed;
    std::vector<std::pair<Entry, Entry>> changed;
    std::vector<std::string> keys_added;
    std::vector<std::string> keys_removed;
};
```

`Snapshot` — плоское представление всех значений и ключей на момент чтения. `Diff` — результат сравнения двух снимков. Ключом в `values` служит строка `<ключ>/<имя значения>`, что упрощает сравнение.

Файл значения имеет вид:

```
REG_SZ
text with \n escaped newline
```

Для `REG_MULTI_SZ` строки разделяются символом перевода строки, для `REG_BINARY` хранится шестнадцатеричная строка, для `REG_DWORD` и `REG_QWORD` — десятичное число. Символы `\`, `\n`, `\r` экранируются.

# 7. СХЕМЫ И ДИАГРАММЫ

Диаграммы выполнены в Graphviz и находятся в редактируемых файлах:

- `diagrams/architecture.dot` — архитектура;
- `diagrams/activity_monitor.dot` — алгоритм монитора.

Перерисовка выполняется скриптом `scripts/render_diagrams.py` или командой `dot -Tpng ...`. Исходные описания и готовые PNG/PDF находятся в репозитории.

## 8. ОПИСАНИЕ ОБРАБОТКИ ОШИБОК

Каждый системный вызов проверяется. Для ошибок POSIX выводится имя функции, выполняемая операция, код `errno` и текст `strerror(errno)`. Для логических ошибок (неизвестный тип, отсутствующий ключ, запись вне учебной области) формируется понятное сообщение. Пример:

```
std::string system_error(const std::string& function,
                         const std::string& operation) {
    const int code = errno;
    return function + "() failed while " + operation + ": errno=" +
           std::to_string(code) + " (" + std::strerror(code) + ")";
}
```

Запись и удаление защищены проверкой учебной области:

```
bool Registry::is_learning_key(const std::string& key) const {
    const auto parts = split_key(key);
    if (parts.size() < 6) return false;
    return parts[0] == "HKEY_CURRENT_USER" &&
           parts[1] == "Software" &&
           parts[2] == "OSiSP" &&
           parts[3] == "Lab06" &&
           parts[4] == student_tag_ &&
           parts[5].rfind("Run_", 0) == 0;
}
```

При удалении наблюдаемого ключа монитор фиксирует `MONITOR_TARGET_REMOVED`. При переполнении очереди `inotify` в журнал записывается `IN_Q_OVERFLOW`, после чего выполняется повторное чтение снимка.

## 9. ТЕСТОВЫЕ СЦЕНАРИИ

Автоматические проверки запускаются через CTest:

```
ctest --test-dir build --output-on-failure
```

| **Сценарий** | **Что проверяется** |
|---|---|
| Создание и список | ключ и значения в `list` |
| Изменение значения | смена типа и данных |
| Удаление значения | исчезновение из снимка |
| Запись вне области | ошибка `write denied` |
| Неверный тип | сообщение об ошибке |
| Неверный `REG_DWORD` | проверка формата числа |
| Отсутствующий ключ | ошибка монитора |
| Изменение одного значения | `REG_NOTIFY` + `REG_MODIFY` |
| Изменение пяти значений | `changed=5` за одну серию |
| Удаление значения | `removed=1` + `REG_DEL` |

Рисунок 2. Тестовые сценарии

Все проверки используют временный корень, поэтому не затрагивают данные пользователя. Результат финального запуска: 6 из 6 групп проверок пройдено, 24 экспериментальных запуска завершились без ошибок.

## 10. МЕТОДИКА ЭКСПЕРИМЕНТОВ

Для каждой конфигурации выполнялось по три запуска. Использовался фиксированный `SEED = 3716`, поэтому набор исходных значений воспроизводим. Монитор запускался с `--max-events 1`; после появления ready-файла скрипт изменял значения через ту же утилиту. Время обнаружения вычислялось как разница между моментом непосредственно перед первой командой изменения и полем `ts_us` события `REG_NOTIFY`. Время повторного чтения бралось из поля `read_us` события `REG_DIFF`; оно измеряется через `clock_gettime(CLOCK_MONOTONIC)` и не включает задержку запуска процессов.

Конфигурации:

- `values_1`, `values_10`, `values_100`, `values_500` — один ключ содержит соответственно 1, 10, 100 и 500 значений; изменяется одно значение;
- `change_1`, `change_5`, `change_10` — изменяется 1, 5 и 10 значений;
- `delete_1` — удаляется одно значение.

## 11. ЭКСПЕРИМЕНТАЛЬНЫЕ РЕЗУЛЬТАТЫ

| Конфигурация | Запуск 1 | Запуск 2 | Запуск 3 | Среднее |
|---|---|---|---|---|
| values_1 | 2.984 | 3.656 | 3.444 | 3.361 |
| values_10 | 3.125 | 3.234 | 2.998 | 3.119 |
| values_100 | 3.065 | 3.527 | 3.017 | 3.203 |
| values_500 | 2.832 | 3.275 | 2.560 | 2.889 |
| change_1 | 2.826 | 2.769 | 3.383 | 2.993 |
| change_5 | 2.603 | 2.650 | 2.869 | 2.707 |
| change_10 | 2.763 | 2.703 | 2.723 | 2.730 |
| delete_1 | 2.494 | 3.057 | 2.342 | 2.631 |

Рисунок 3. Задержка уведомления, мс

| Конфигурация | Запуск 1 | Запуск 2 | Запуск 3 | Среднее |
|---|---|---|---|---|
| values_1 | 0.291 | 0.229 | 0.277 | 0.266 |
| values_10 | 0.480 | 0.386 | 0.474 | 0.447 |
| values_100 | 2.490 | 2.216 | 2.390 | 2.365 |
| values_500 | 8.479 | 8.153 | 8.096 | 8.243 |

Рисунок 4. Время повторного чтения снимка, мс

| Конфигурация | Запрошено | Обнаружено | Уведомлений | Ошибок | Статус |
|---|---|---|---|---|---|
| values_1 | 1 | 1 | 1 | 0 | OK |
| values_10 | 1 | 1 | 1 | 0 | OK |
| values_100 | 1 | 1 | 1 | 0 | OK |
| values_500 | 1 | 1 | 1 | 0 | OK |
| change_1 | 1 | 1 | 1 | 0 | OK |
| change_5 | 5 | 5 | 1 | 0 | OK |
| change_10 | 10 | 10 | 1 | 0 | OK |
| delete_1 | 1 | 1 | 1 | 0 | OK |

Рисунок 5. Профильные показатели варианта

# 12. ГРАФИКИ И АНАЛИЗ

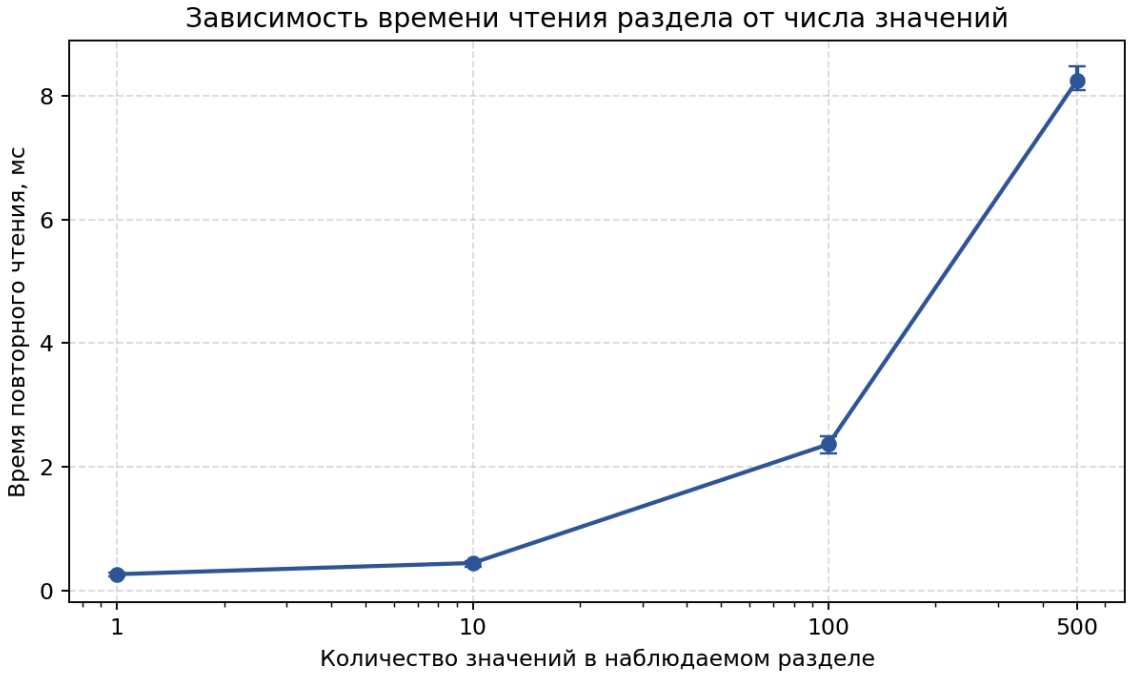



Рисунок 3. Зависимость времени повторного чтения от числа значений

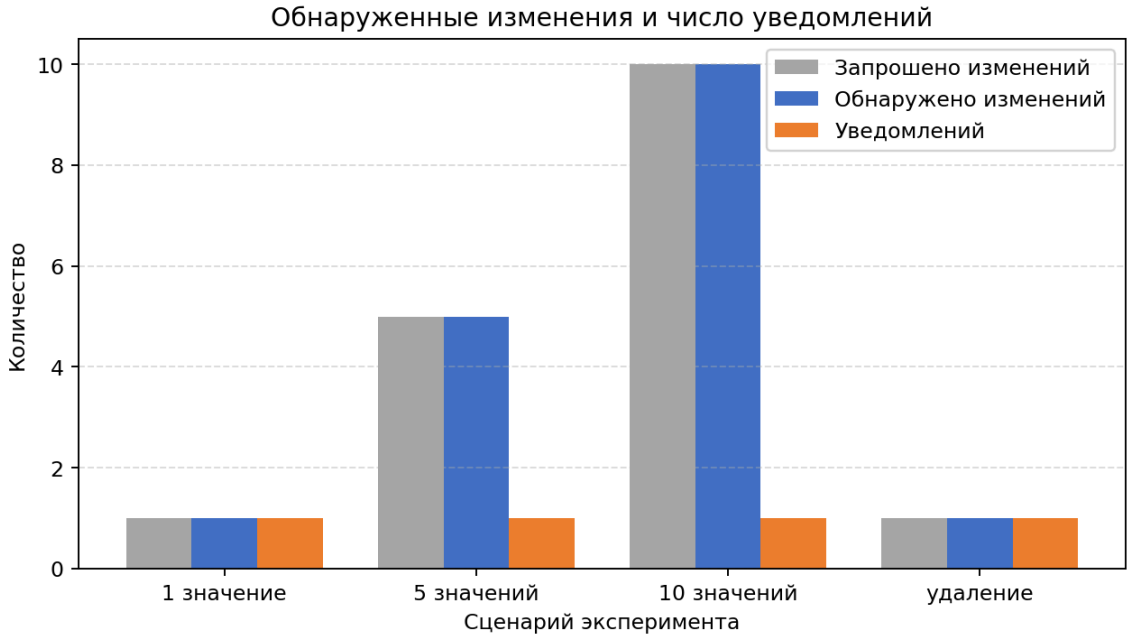



Рисунок 4. Обнаруженные изменения и число уведомлений

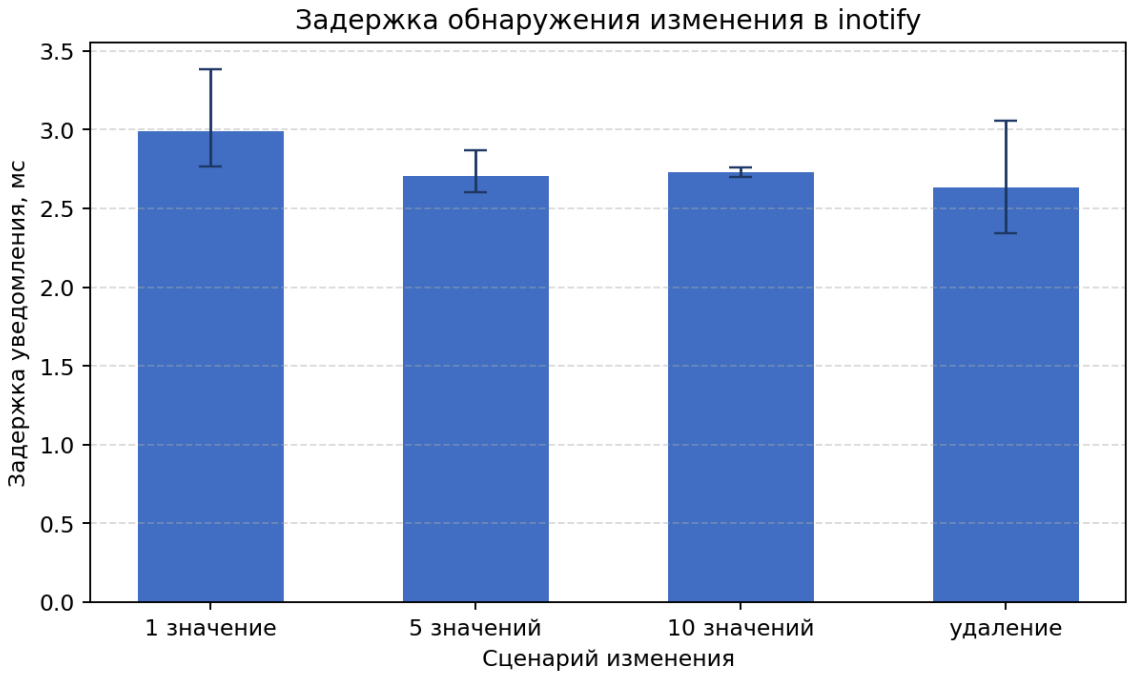



Рисунок 5. Задержка обнаружения изменения

Время повторного чтения растёт почти линейно при увеличении числа значений: на одном значении среднее составило 0,266 мс, на 500 значениях — 8,243 мс. Это ожидаемо: после каждого уведомления программа обходит каталог, открывает и разбирает все файлы значений, а затем строит разностный снимок. Основная стоимость приходится на файловые операции, а не на сам `inotify`.

Задержка уведомления во всех конфигурациях остаётся около 2,6–3,4 мс и почти не зависит от числа изменённых значений. Причина в том, что `inotify` сообщает о событии сразу после записи в файл, а измерение фиксирует именно момент получения события. Повторное чтение и сравнение выполняются после этого и в данную задержку не входят.

Серия из 5 и 10 изменений объединяется окном `--debounce-ms 300` в одно уведомление, но `REG_DIFF` корректно показывает 5 и 10 изменённых значений. Это демонстрирует важное свойство механизма: уведомление сообщает о факте изменения, а количество и характер изменений определяются повторным чтением. При удалении значения формируется `REG_DEL` с именем и старым значением.

Ограничения: `inotify` не хранит историю изменений, поэтому при переполнении очереди возможна потеря промежуточных состояний; итоговое состояние всё равно определяется снимком. Стоимость повторного чтения растёт с числом значений в выбранном ключе.

# 13. ЗАКЛЮЧЕНИЕ

В ходе работы реализована утилита, которая воспроизводит служебную задачу мониторинга изменений значений на Ubuntu. Изучены иерархическое хранилище ключей, типы значений, атомарная запись, журналирование и событийное ожидание через `inotify`. Программа корректно определяет изменение одного и нескольких значений, а также удаление значения, обрабатывает ошибки и освобождает ресурсы. Эксперименты подтвердили, что задержка уведомления слабо зависит от объёма изменений, а стоимость повторного чтения растёт с числом значений.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ


1. Microsoft Learn. Registry Functions. URL: https://learn.microsoft.com/windows/win32/sysinfo/registry-functions
2. Microsoft Learn. RegNotifyChangeKeyValue function. URL: https://learn.microsoft.com/windows/win32/api/winreg/nf-winreg-regnotifychangekeyvalue
3. Linux man-pages. inotify(7). URL: https://man7.org/linux/man-pages/man7/inotify.7.html
4. Linux man-pages. poll(2). URL: https://man7.org/linux/man-pages/man2/poll.2.html
5. Linux man-pages. clock_gettime(2). URL: https://man7.org/linux/man-pages/man2/clock_gettime.2.html
6. cppreference. std::filesystem. URL: https://en.cppreference.com/w/cpp/filesystem

# ПРИЛОЖЕНИЕ А. ИСПОЛЬЗОВАНИЕ ИИ

Случай использования № 1.

**Инструмент:** opencode (CLI-агент).

**Модель:** `opencode/deepseek-v4.1-flash-free`; точная версия в интерфейсе не отображается.

**Цель использования:** подготовка каркаса проекта, реализация файлового аналога реестра, монитора `inotify`, тестов, скриптов экспериментов и отчёта по варианту 6.

**Оригинальный промпт:** полный текст приведён в `report/ai_prompts.md`.

**Полученный результат:** исходный код на C++17, CMake-файл, Python-проверки, скрипты экспериментов и графиков, диаграммы Graphviz и черновик отчёта.

**Место использования:** `src/registry.cpp`, `src/monitor.cpp`, `src/main.cpp`, `tests/run_checks.py`, `scripts/run_experiments.py`, `scripts/plot_results.py`.

**Самостоятельная проверка:** сборка проекта, запуск CTest, ручная проверка изменения и удаления значений, анализ журналов `REG_NOTIFY`/`REG_DIFF`, повторный запуск экспериментов и сравнение чисел в таблицах с CSV.

**Самостоятельные изменения:** выбраны имена и структура команд, учебный каталог, формат хранения значений, окно объединения событий, набор экспериментальных конфигураций и оформление отчёта.

## ПРИЛОЖЕНИЕ Б. СВЕДЕНИЯ О РЕПОЗИТОРИИ GIT

- URL: https://github.com/Dima-Vasilenko/osisp
- Каталог: `lab6/`
- Ветка: `main`
- Основные коммиты: каркас, реализация реестра, реализация монитора, тесты, эксперименты, графики, отчёт.
- Полный исходный код, данные, скрипты и редактируемые диаграммы находятся в репозитории.

## ПРИЛОЖЕНИЕ В. ЛИСТИНГ КОДА ПРОГРАММЫ

Фрагмент атомарной записи значения:

```
std::string pattern = (fs::path(dir) / ".tmpXXXXXX").string();
std::vector<char> buffer(pattern.begin(), pattern.end());
buffer.push_back('\0');
const int fd = mkstemp(buffer.data());
if (fd < 0) { /* ошибка */ }
const std::string temp_path(buffer.data());
const std::string content = checked.type + "\n" + escape_text(checked.data);
if (!write_all(fd, content)) { /* ошибка и unlink */ }
if (fsync(fd) != 0) { /* ошибка и unlink */ }
close(fd);
const std::string target = (fs::path(dir) / (name + ".val")).string();
if (rename(temp_path.c_str(), target.c_str()) != 0) { /* ошибка */ }
```

Фрагмент регистрации наблюдения:

```
const int fd = inotify_init1(IN_CLOEXEC);
const int wd = inotify_add_watch(fd, dir.c_str(), WATCH_MASK);
watches[wd] = key;
```

Фрагмент ожидания и сравнения:

```
pollfd descriptor{fd, POLLIN, 0};
const int ready = poll(&descriptor, 1, wait_ms);
if (ready > 0) {
    read_events(fd, event_count, all_mask, relevant, error);
}
Snapshot current;
registry.read_snapshot(config.key, false, current, error);
const Diff diff = diff_snapshots(previous, current);
previous = current;
```

Полный листинг находится в файлах `src/*.cpp` и `src/*.h` репозитория.

# ПРИЛОЖЕНИЕ Г. ЛИСТИНГ КОДА ГРАФИЧЕСКИХ МАТЕРИАЛОВ

Графики строятся скриптом `scripts/plot_results.py` с использованием `matplotlib`. Он читает `data/raw/experiment_raw.csv` и сохраняет PNG/PDF в `data/plots/`. Диаграммы строятся из `diagrams/*.dot` через Graphviz.

Команды воспроизведения:

```
python3 scripts/run_experiments.py
python3 scripts/plot_results.py
python3 scripts/render_diagrams.py
```

## ПРИЛОЖЕНИЕ Д. ИСХОДНЫЕ ЭКСПЕРИМЕНТАЛЬНЫЕ ДАННЫЕ

- `data/raw/experiment_raw.csv` — 24 запуска с задержками, временем чтения, количеством обнаруженных изменений и статусом;
- `data/raw/summary.csv` — средние, минимальные и максимальные значения по конфигурациям;
- `data/raw/logs/` — журналы монитора для каждого запуска.